

# **ALFIDO TECH INTERNSHIP**

## **Task 1: Java Basics & OOP Fundamentals**

Submitted By: Pravallika Eichapuram

Internship: Java Developer Internship

Task Number: 1

Date: 06 July 2026

### **Objective :**

The objective of this task is to understand the fundamentals of Java programming and Object-Oriented Programming (OOP). This task demonstrates the implementation of loops, conditions, arrays, classes, objects, inheritance, polymorphism, and encapsulation through simple Java programs.

### **Programs Implemented :**

1. Bank Management System
2. Loops, Conditions and Arrays

### **Program 1: Bank Management System :**

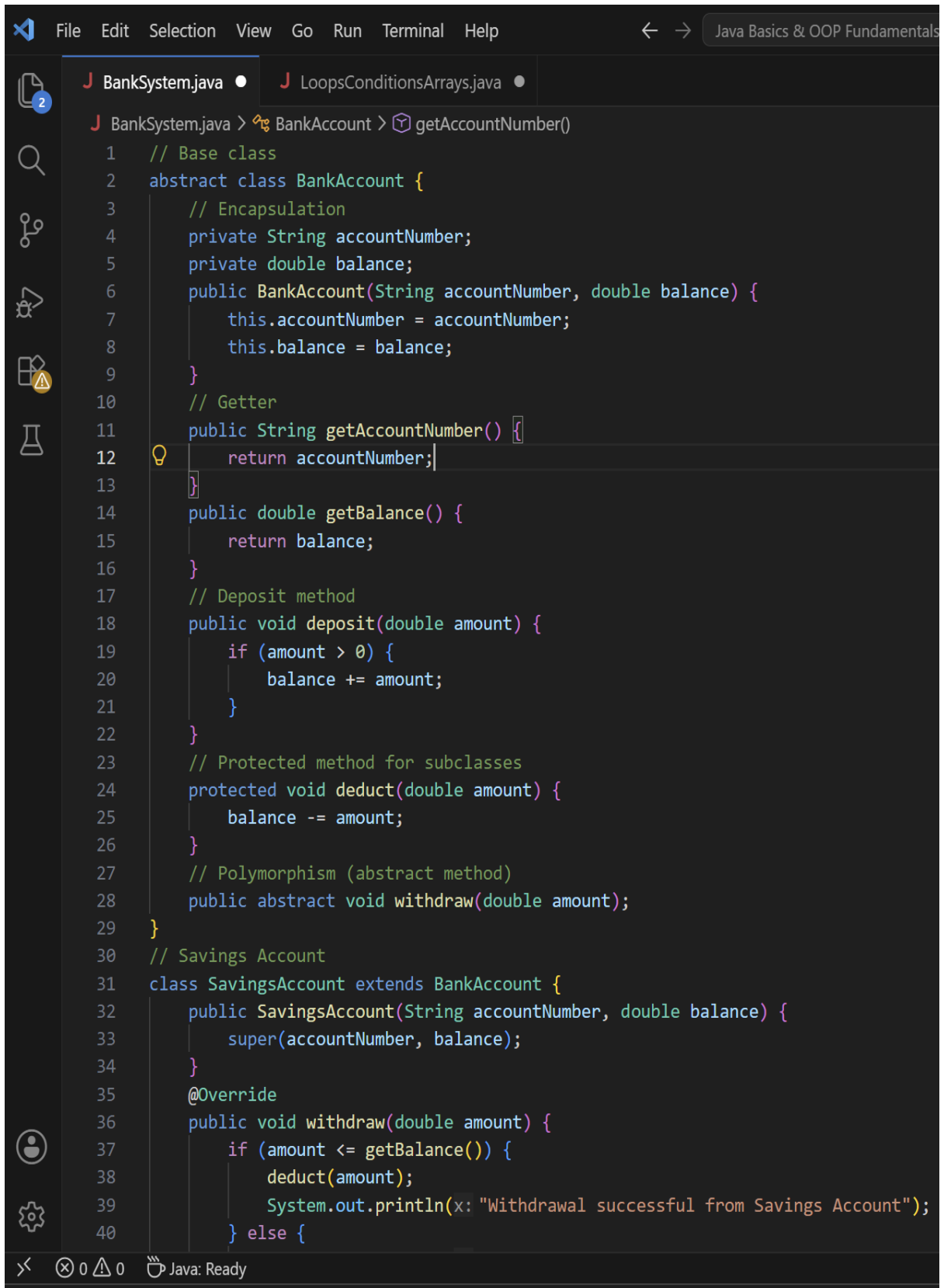
#### **Description :**

This program demonstrates the basic Object-Oriented Programming concepts in Java by implementing a simple Bank Management System.

#### **Features :**

- Uses classes and objects
- Demonstrates encapsulation
- Demonstrates inheritance
- Uses abstraction
- Performs deposit and withdrawal operations

## Source Code :

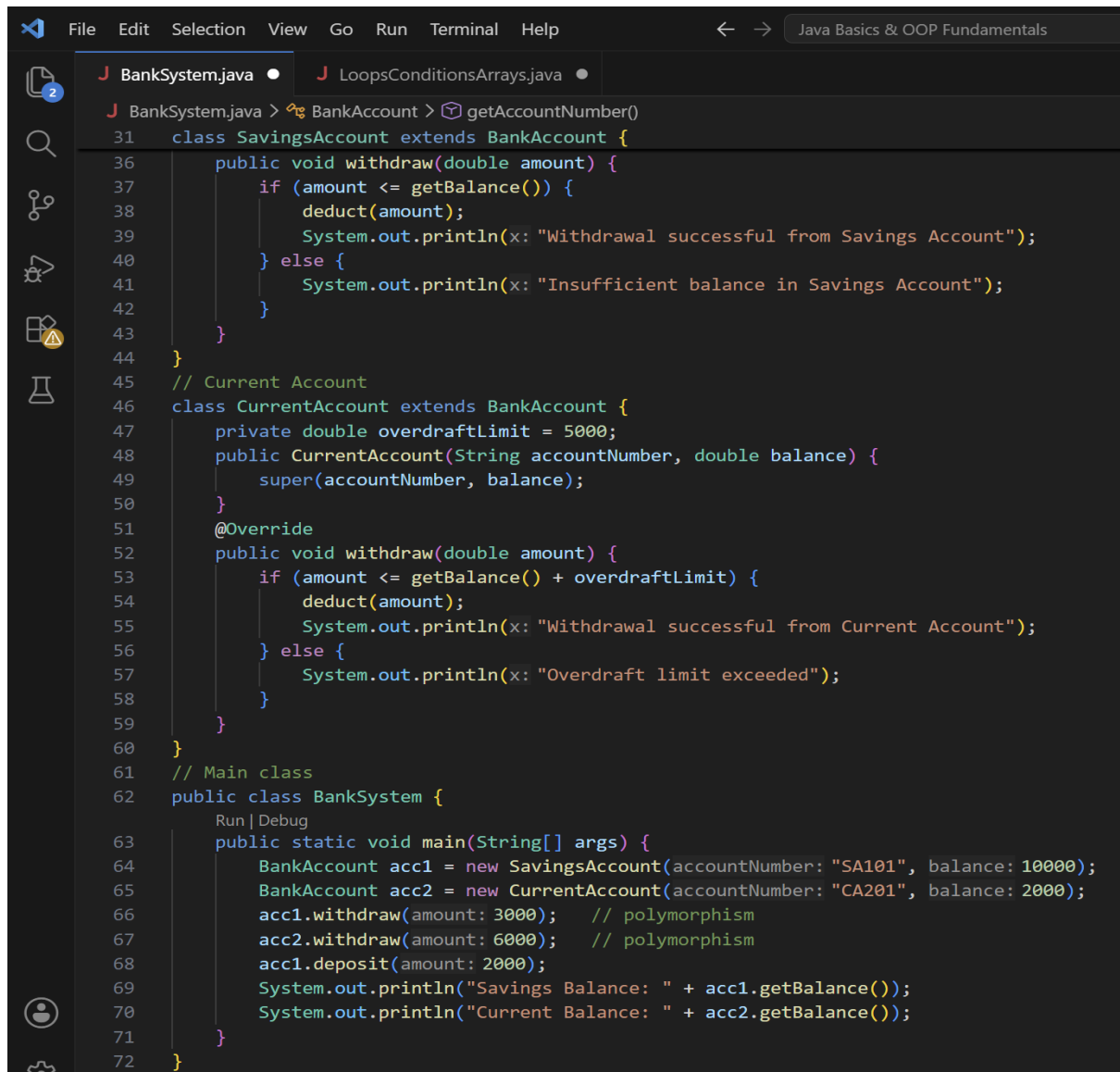


```
File Edit Selection View Go Run Terminal Help
Java Basics & OOP Fundamentals

J BankSystem.java • J LoopsConditionsArrays.java •

J BankSystem.java > BankAccount > getAccountNumber()

1 // Base class
2 abstract class BankAccount {
3     // Encapsulation
4     private String accountNumber;
5     private double balance;
6     public BankAccount(String accountNumber, double balance) {
7         this.accountNumber = accountNumber;
8         this.balance = balance;
9     }
10    // Getter
11    public String getAccountNumber() {}
12    return accountNumber;
13 }
14 public double getBalance() {
15     return balance;
16 }
17 // Deposit method
18 public void deposit(double amount) {
19     if (amount > 0) {
20         balance += amount;
21     }
22 }
23 // Protected method for subclasses
24 protected void deduct(double amount) {
25     balance -= amount;
26 }
27 // Polymorphism (abstract method)
28 public abstract void withdraw(double amount);
29 }
30 // Savings Account
31 class SavingsAccount extends BankAccount {
32     public SavingsAccount(String accountNumber, double balance) {
33         super(accountNumber, balance);
34     }
35     @Override
36     public void withdraw(double amount) {
37         if (amount <= getBalance()) {
38             deduct(amount);
39             System.out.println(x: "Withdrawal successful from Savings Account");
40         } else {
```



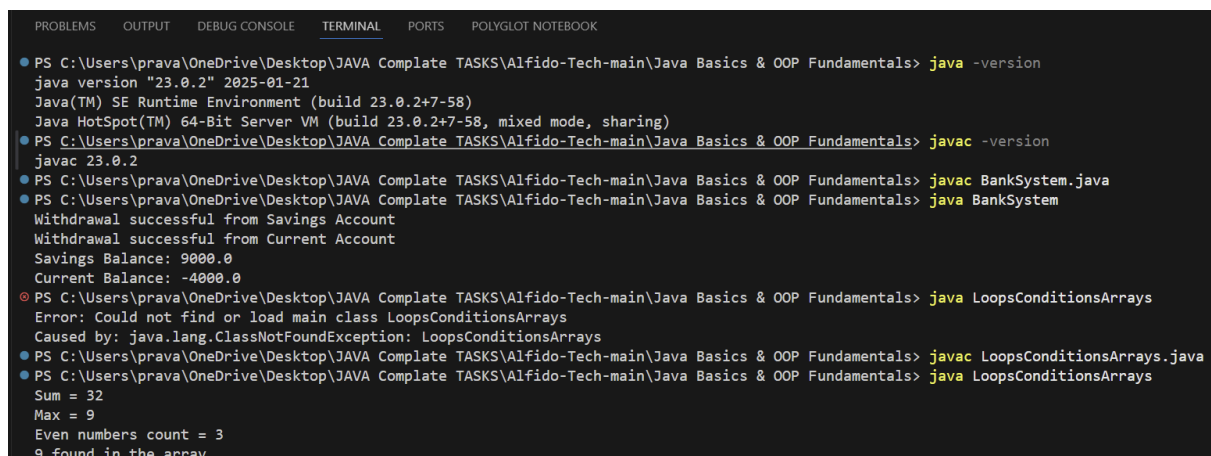
```
File Edit Selection View Go Run Terminal Help
Java Basics & OOP Fundamentals

J BankSystem.java
J LoopsConditionsArrays.java

J BankSystem.java > BankAccount > getAccountNumber()
31 class SavingsAccount extends BankAccount {
36     public void withdraw(double amount) {
37         if (amount <= getBalance()) {
38             deduct(amount);
39             System.out.println(x: "Withdrawal successful from Savings Account");
40         } else {
41             System.out.println(x: "Insufficient balance in Savings Account");
42         }
43     }
44 }
45 // Current Account
46 class CurrentAccount extends BankAccount {
47     private double overdraftLimit = 5000;
48     public CurrentAccount(String accountNumber, double balance) {
49         super(accountNumber, balance);
50     }
51     @Override
52     public void withdraw(double amount) {
53         if (amount <= getBalance() + overdraftLimit) {
54             deduct(amount);
55             System.out.println(x: "Withdrawal successful from Current Account");
56         } else {
57             System.out.println(x: "Overdraft limit exceeded");
58         }
59     }
60 }
61 // Main class
62 public class BankSystem {
63     Run | Debug
64     public static void main(String[] args) {
65         BankAccount acc1 = new SavingsAccount(accountNumber: "SA101", balance: 10000);
66         BankAccount acc2 = new CurrentAccount(accountNumber: "CA201", balance: 2000);
67         acc1.withdraw(amount: 3000); // polymorphism
68         acc2.withdraw(amount: 6000); // polymorphism
69         acc1.deposit(amount: 2000);
70         System.out.println("Savings Balance: " + acc1.getBalance());
71         System.out.println("Current Balance: " + acc2.getBalance());
72     }
73 }
```

Figure 1: BankSystem.java Source Code

## Output :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK

PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> java -version
java version "23.0.2" 2025-01-21
Java(TM) SE Runtime Environment (build 23.0.2+7-58)
Java HotSpot(TM) 64-Bit Server VM (build 23.0.2+7-58, mixed mode, sharing)
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> javac -version
javac 23.0.2
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> javac BankSystem.java
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> java BankSystem
Withdrawal successful from Savings Account
Withdrawal successful from Current Account
Savings Balance: 9000.0
Current Balance: -4000.0
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> java LoopsConditionsArrays
Error: Could not find or load main class LoopsConditionsArrays
Caused by: java.lang.ClassNotFoundException: LoopsConditionsArrays
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> javac LoopsConditionsArrays.java
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> java LoopsConditionsArrays
Sum = 32
Max = 9
Even numbers count = 3
9 found in the array.
```

Figure 2: BankSystem Output

## Program 2: Loops, Conditions and Arrays :

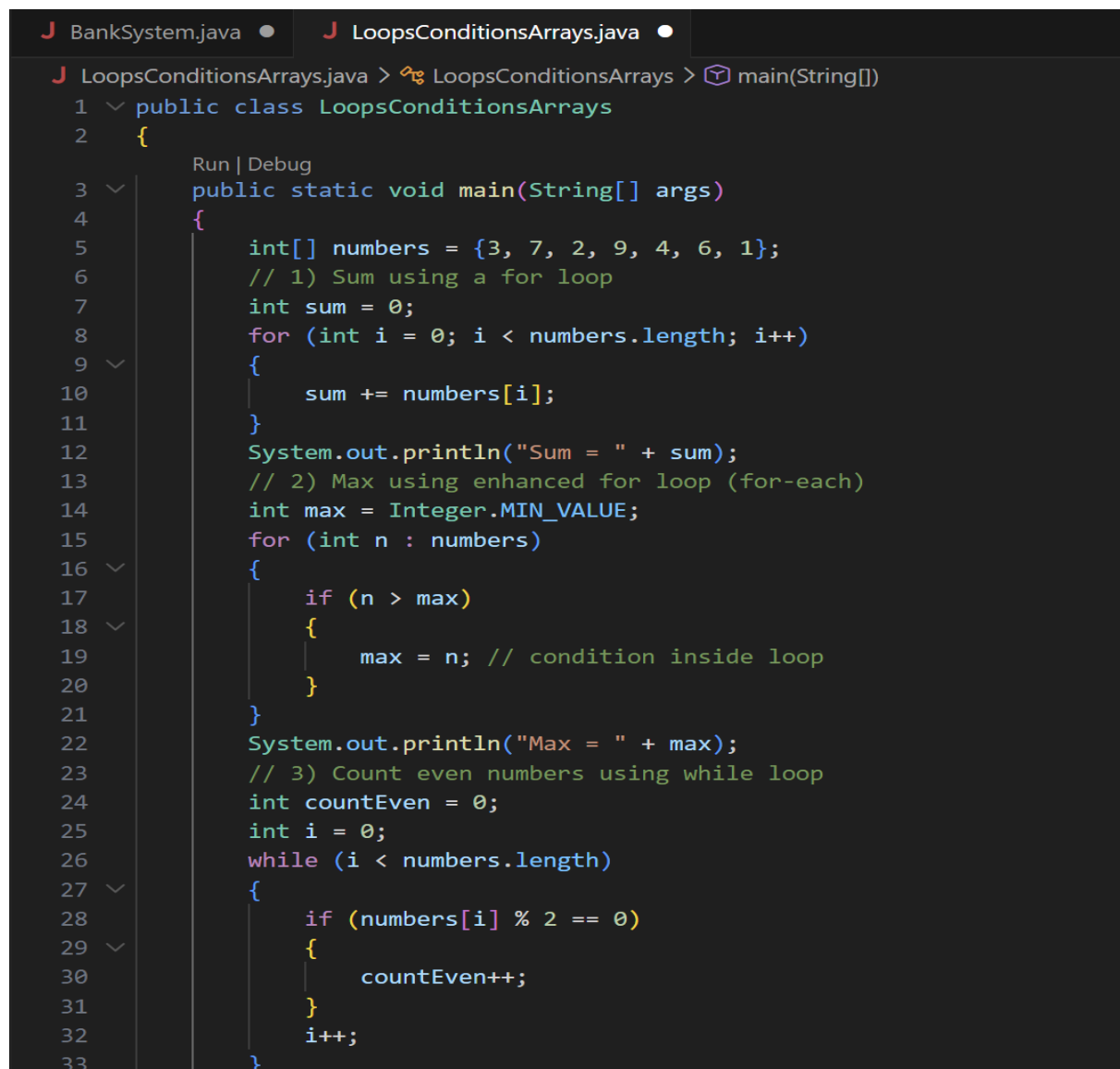
### Description :

This program demonstrates Java control statements and array operations.

### Features :

- Uses for loop
- Uses while loop
- Uses if-else statements
- Finds the sum of array elements
- Finds the maximum value
- Counts even numbers
- Searches for a particular element

### Source Code :



```
BankSystem.java • LoopsConditionsArrays.java •
LoopsConditionsArrays.java > LoopsConditionsArrays > main(String[])
1  public class LoopsConditionsArrays
2  {
3      Run | Debug
4      public static void main(String[] args)
5      {
6          int[] numbers = {3, 7, 2, 9, 4, 6, 1};
7          // 1) Sum using a for loop
8          int sum = 0;
9          for (int i = 0; i < numbers.length; i++)
10         {
11             sum += numbers[i];
12         }
13         System.out.println("Sum = " + sum);
14         // 2) Max using enhanced for loop (for-each)
15         int max = Integer.MIN_VALUE;
16         for (int n : numbers)
17         {
18             if (n > max)
19             {
20                 max = n; // condition inside loop
21             }
22         }
23         System.out.println("Max = " + max);
24         // 3) Count even numbers using while loop
25         int countEven = 0;
26         int i = 0;
27         while (i < numbers.length)
28         {
29             if (numbers[i] % 2 == 0)
30             {
31                 countEven++;
32             }
33             i++;
34         }
35     }
36 }
```

```
J BankSystem.java • J LoopsConditionsArrays.java •
J LoopsConditionsArrays.java > LoopsConditionsArrays > main(String[])
1 public class LoopsConditionsArrays
3     public static void main(String[] args)
32         // 4) Search for a value with if/else
33     }
34     System.out.println("Even numbers count = " + countEven);
35     // 4) Search for a value with if/else
36     int target = 9;
37     boolean found = false;
38     for (int val : numbers)
39     {
40         if (val == target)
41         {
42             found = true;
43             break;
44         }
45     }
46     if (found)
47     {
48         System.out.println(target + " found in the array.");
49     }
50     else
51     {
52         System.out.println(target + " NOT found in the array.");
53     }
54 }
55 }
```

Figure 3: LoopsConditionsArrays.java Source Code

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> java -version
java version "23.0.2" 2025-01-21
Java(TM) SE Runtime Environment (build 23.0.2+7-58)
Java HotSpot(TM) 64-Bit Server VM (build 23.0.2+7-58, mixed mode, sharing)
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> javac -version
javac 23.0.2
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> javac BankSystem.java
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> java BankSystem
Withdrawal successful from Savings Account
Withdrawal successful from Current Account
Savings Balance: 9000.0
Current Balance: -4000.0
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> java LoopsConditionsArrays
Error: Could not find or load main class LoopsConditionsArrays
Caused by: java.lang.ClassNotFoundException: LoopsConditionsArrays
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> javac LoopsConditionsArrays.java
PS C:\Users\prava\OneDrive\Desktop\JAVA Complate TASKS\Alfido-Tech-main\Java Basics & OOP Fundamentals> java LoopsConditionsArrays
Sum = 32
Max = 9
Even numbers count = 3
9 found in the array.
```

Figure 4: Program Output

## **OOP Concepts Used :**

### **Encapsulation :**

Encapsulation protects data by declaring variables as private and providing public getter and setter methods to access them.

### **Inheritance :**

Inheritance allows one class to acquire the properties and methods of another class. In this task, the SavingsAccount and CurrentAccount classes inherit from the BankAccount class.

### **Polymorphism :**

Polymorphism allows the same method to perform different actions. The withdraw() method is overridden in different account classes.

### **Abstraction :**

Abstraction hides implementation details and exposes only essential functionality. The BankAccount class is implemented as an abstract class.

## **Technologies Used :**

- Java
- Object-Oriented Programming
- VS Code
- JDK 17/21
- GitHub

## **Conclusion :**

This task successfully demonstrates Java programming fundamentals and Object-Oriented Programming concepts. The programs were compiled and executed successfully, and all required concepts such as loops, arrays, conditions, encapsulation, inheritance, abstraction, and polymorphism were implemented successfully.